Unit 3

COGNITION

27. Turing Machines

Turing's Idea

Turing's problem: can all mathematical proofs be described mechanically? What kind of mechanical tools would be needed to generate any given proof?

This problem arose because of Turing's interest in the **Entscheidungsproblem**, or decision problem: the question of whether there is a general algorithm for determining whether any given mathematical claim is provable.

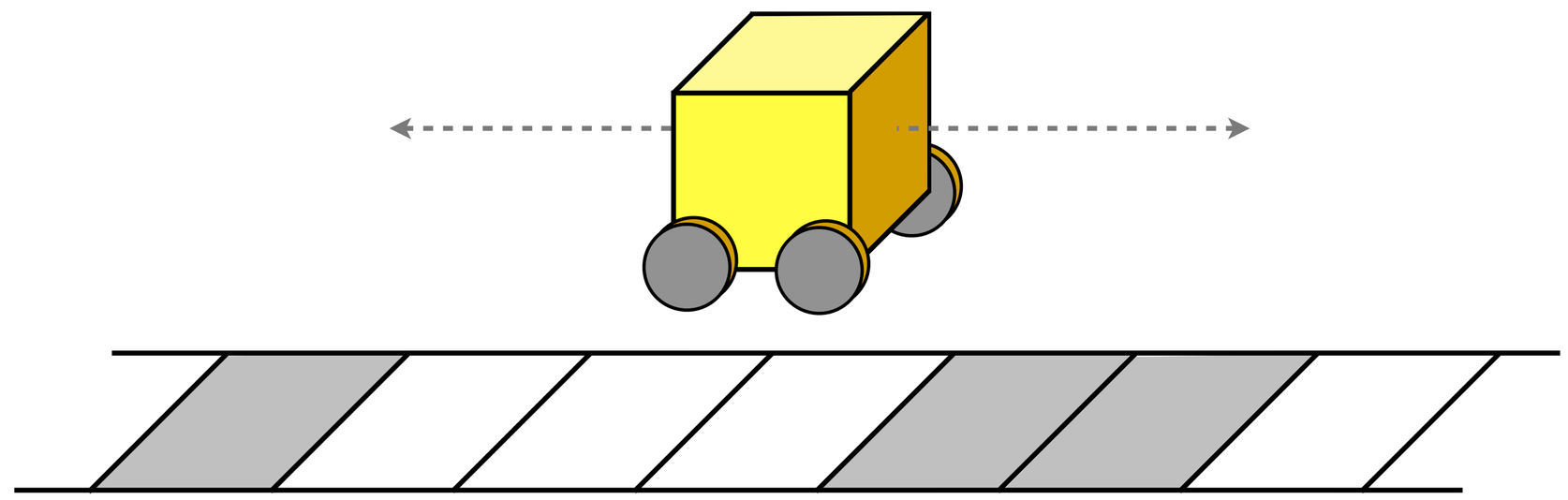
Turing reasoned that the tools you need to solve any given math problem are:

- A piece of paper.
- A stock of mathematical symbols.
- A pencil that writes.
- A human controller.

Now simplify each of these to their simplest mechanical form:

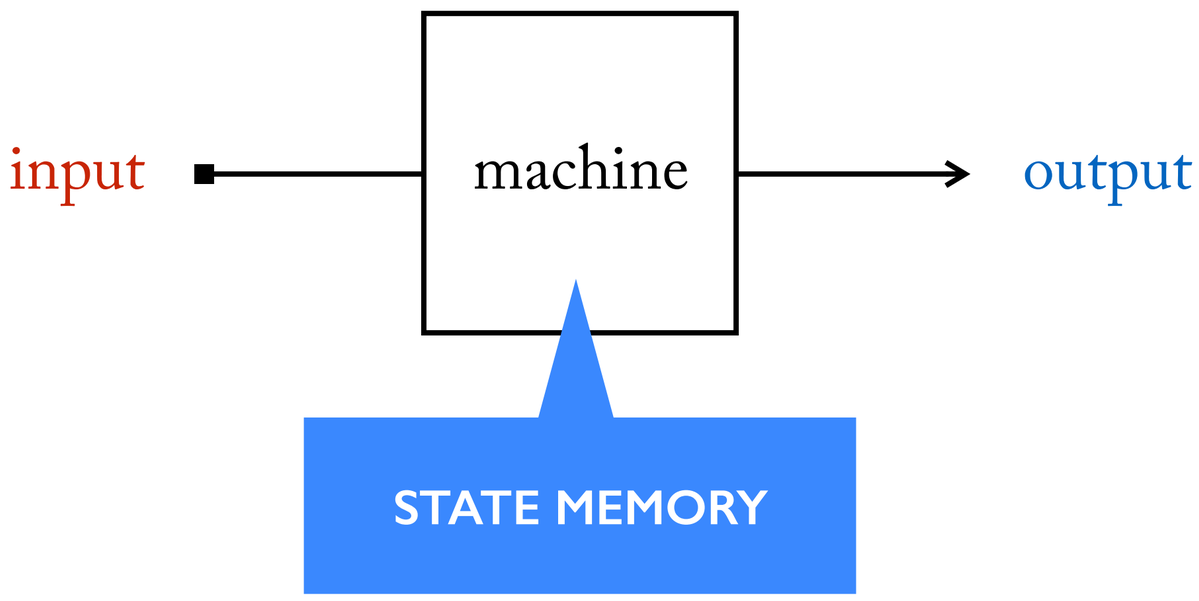
- The paper becomes a 1-dimensional tape.
- The mathematical symbols become 0's and 1's.
- The actions of the pencil become write a 0, write a 1, move left, move right.
- The controller becomes a finite state machine that can read 0's and 1's,

A machine which computes in this way is a Turing Machine (TM).

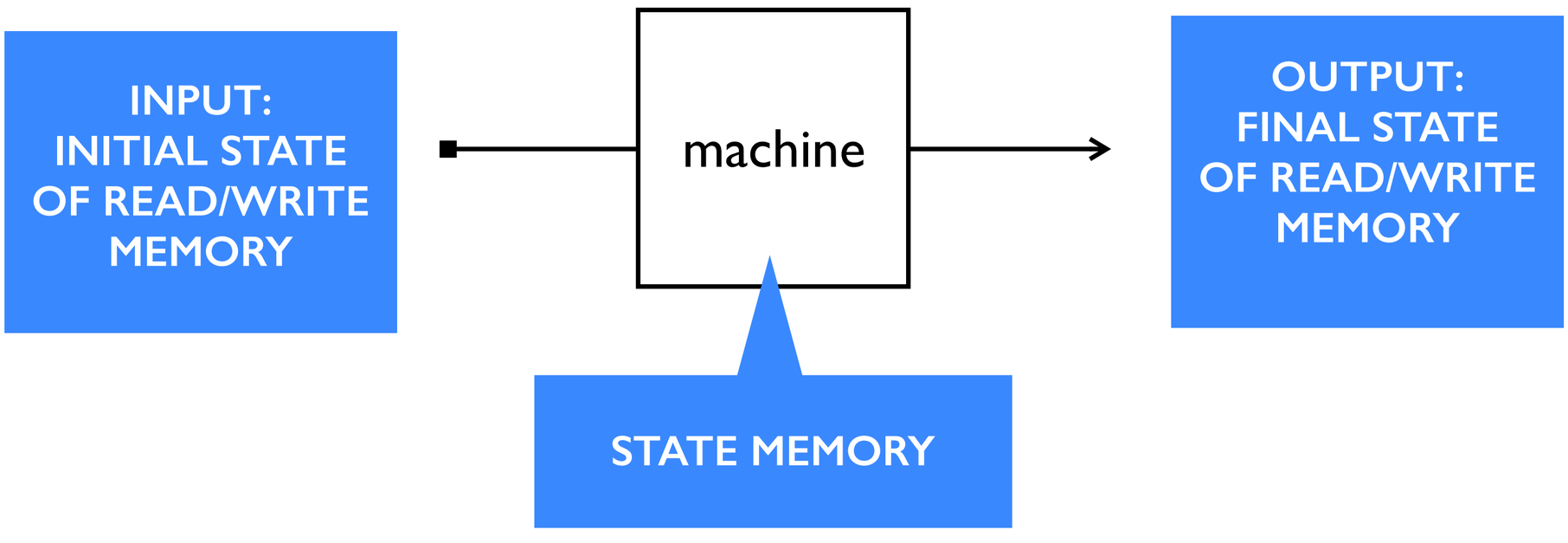


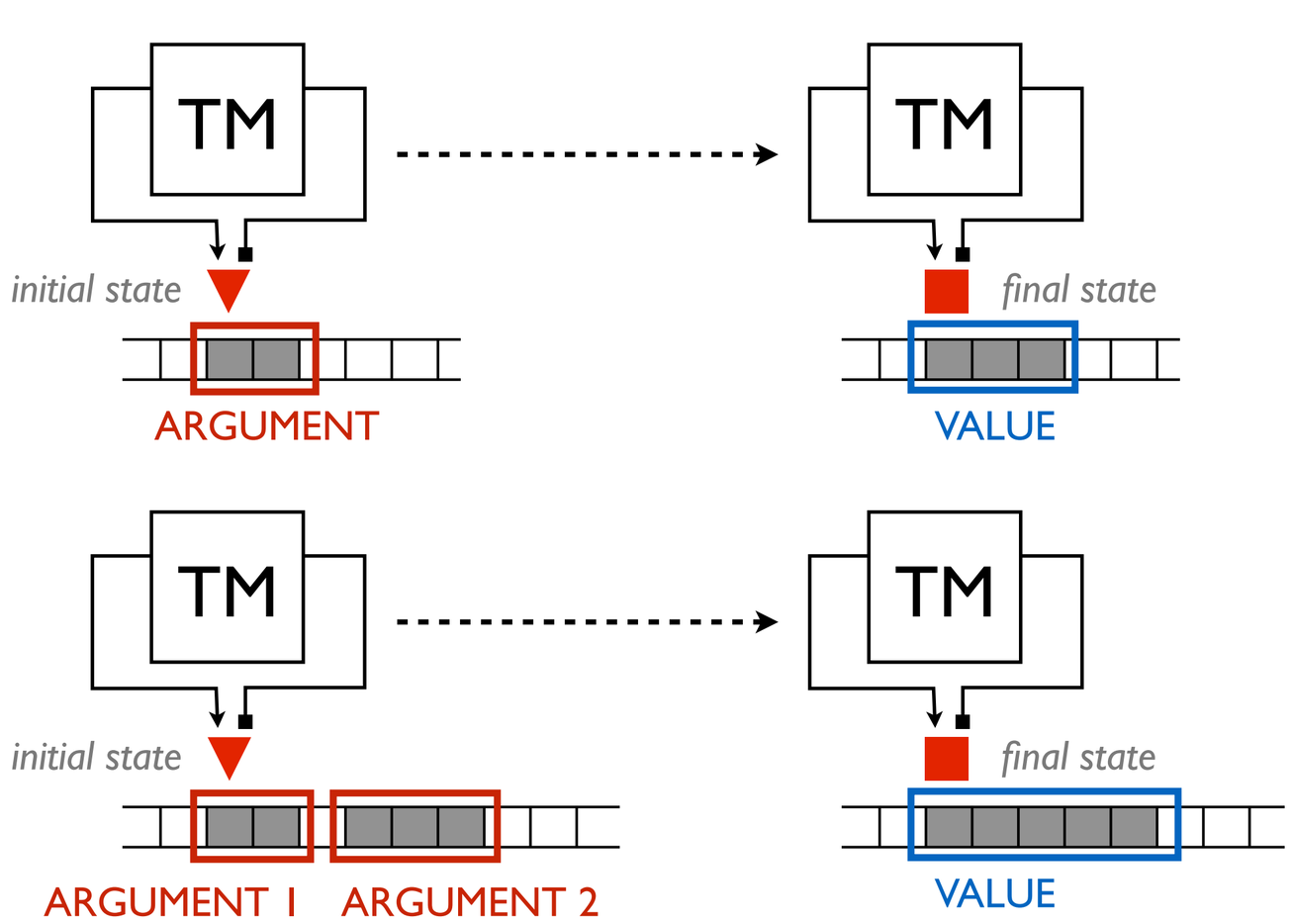
Computing with Memory

FSM: State Memory.
State memory is: (1) short-term; (2) fixed by the architecture of the system.

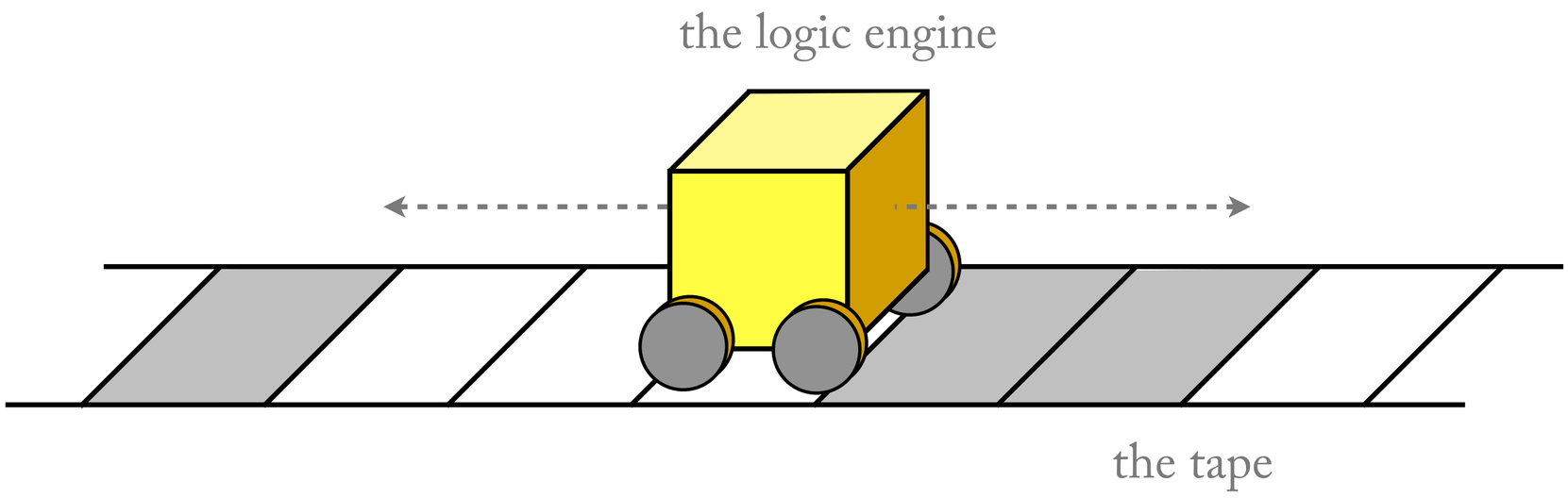


TM: State Memory + Read/Write Memory
Read/write memory is: (1) stable; (2) changeable; (3) extendable within a system.





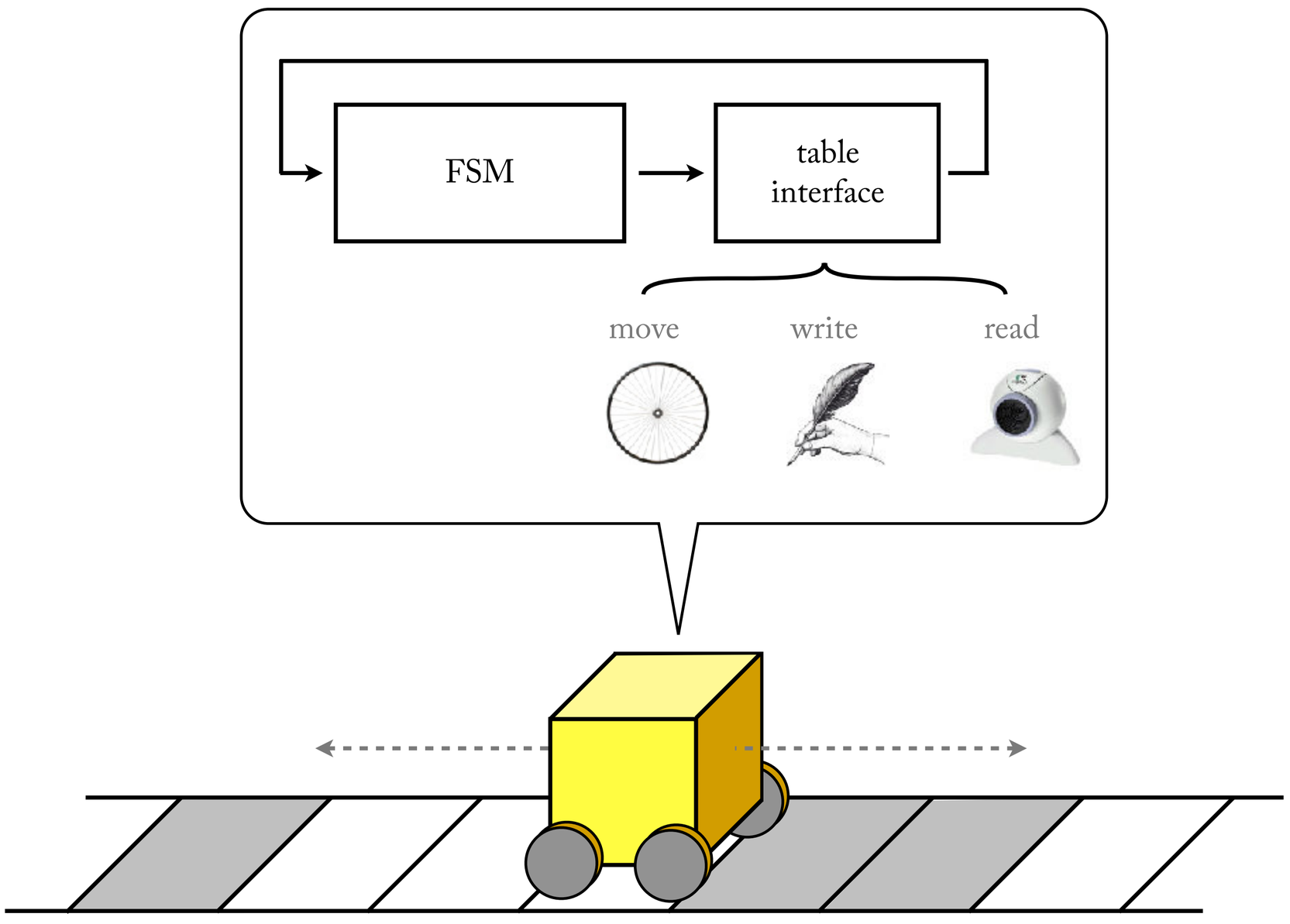
Turing Machine Architecture



A Turing Machine has a two-part architecture:
(i) **A logic engine** (a finite state machine).
(ii) **A read-write memory** (a tape).

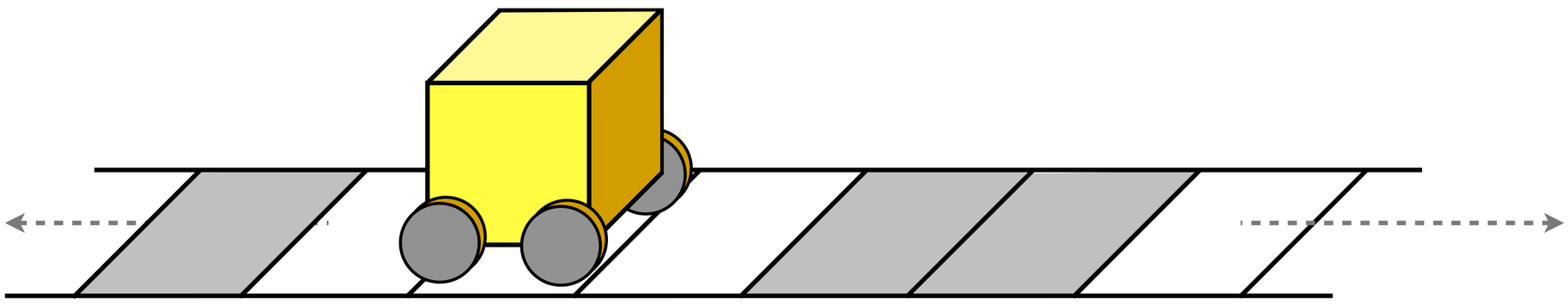
The logic engine is a finite state machine, realized by a sequential circuit. The tape is an infinite strip of graph paper covered with 0's and 1's which the logic engine reads from and writes onto.

In order to work on the tape, the FSM inside the TM is connected to a tape-interface. This interface allows the FSM to move left and right on the tape, write onto it, and read from it. The whole system is connected together in a loop so that the machine calculates, then interacts with the tape, then repeats.



Each reading of the tape becomes the input to the circuit. Each output of the circuit becomes an action of the machine on the tape. The behavior of the machine as a whole depends on both the content of the tape and the content of the memory registers in the sequential circuit.

The Tape



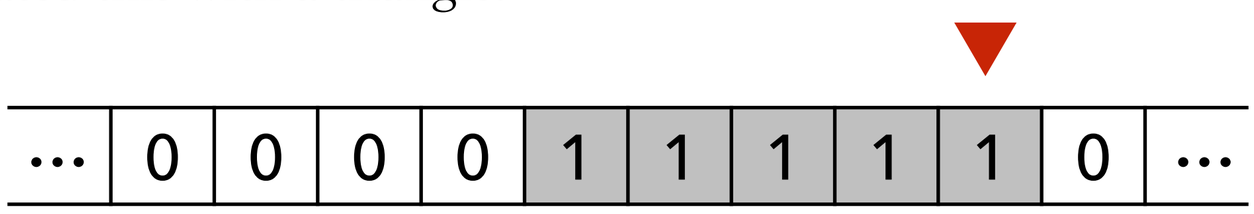
The **turing tape** has the following properties:

- The tape is infinitely long.
- It is divided into cells.
- Every cell is either filled-in (1) or empty (0).
- There are **infinitely** many empty cells (0's) on the tape.
- There are **finitely** many filled-in cells (1's) on the tape.

Machine-Tape Interface

At any given time, the machine occupies exactly one cell of the tape. This is called the **position** of the machine at that time.

We'll indicated this with a triangle:



The machine's abilities are very limited. It can only perform "1-cell" operations: it can only write on, move across, or read one cell at a time.

Actions

The machine can only perform 1-cell actions:

1. **R** = move right one cell.
2. **L** = move left one cell.
3. **1** = make the current cell filled-in.
4. **0** = make the current cell empty.
5. **Halt**

Perceptions

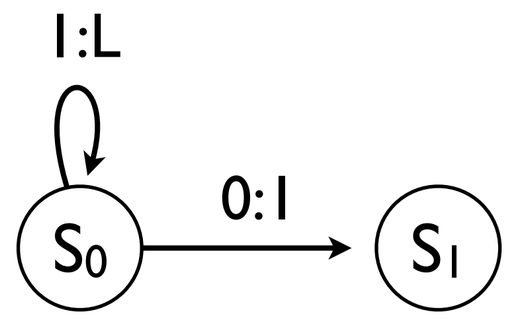
The machine can read only two 1-cell states of the tape:

1. **1** = the current cell is filled-in.
2. **0** = the current cell is empty.

Turing Machine Diagrams

TM diagrams are just like FSM diagrams, where the possible inputs are (for now) 1 or 0, and the possible outputs are R, L, 1, or 0.

This FSM can be realized by a sequential circuit with one input, and several outputs which control a motor.



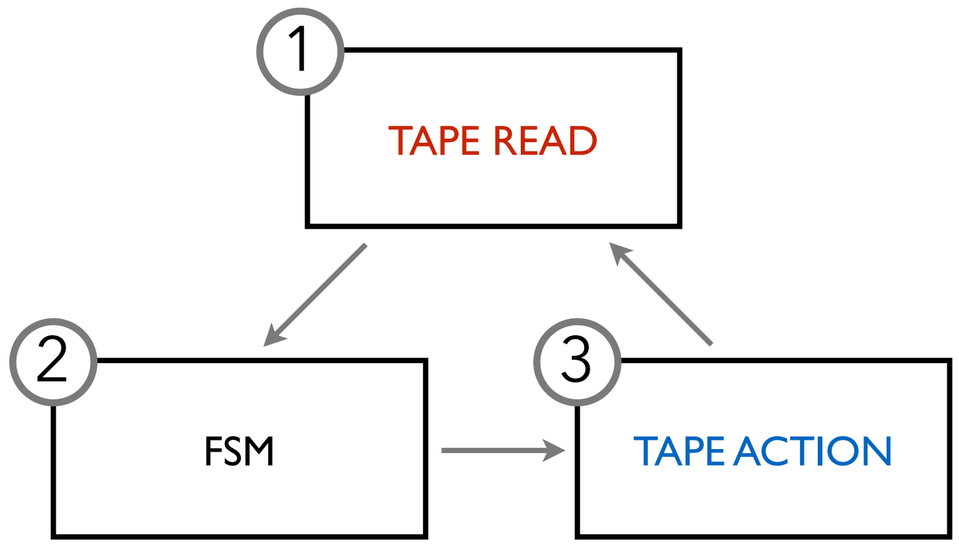
This machine above reads 1's and moves left, until it reads a 0, then writes a 1.

Operation in Time

Order of operations

At each time cycle, the machine undergoes a sequence of actions:

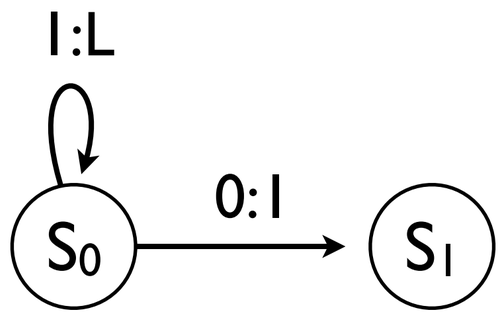
1. Read tape
2. Run circuit
3. Execute tape action (or halt)
4. Repeat



Halting:

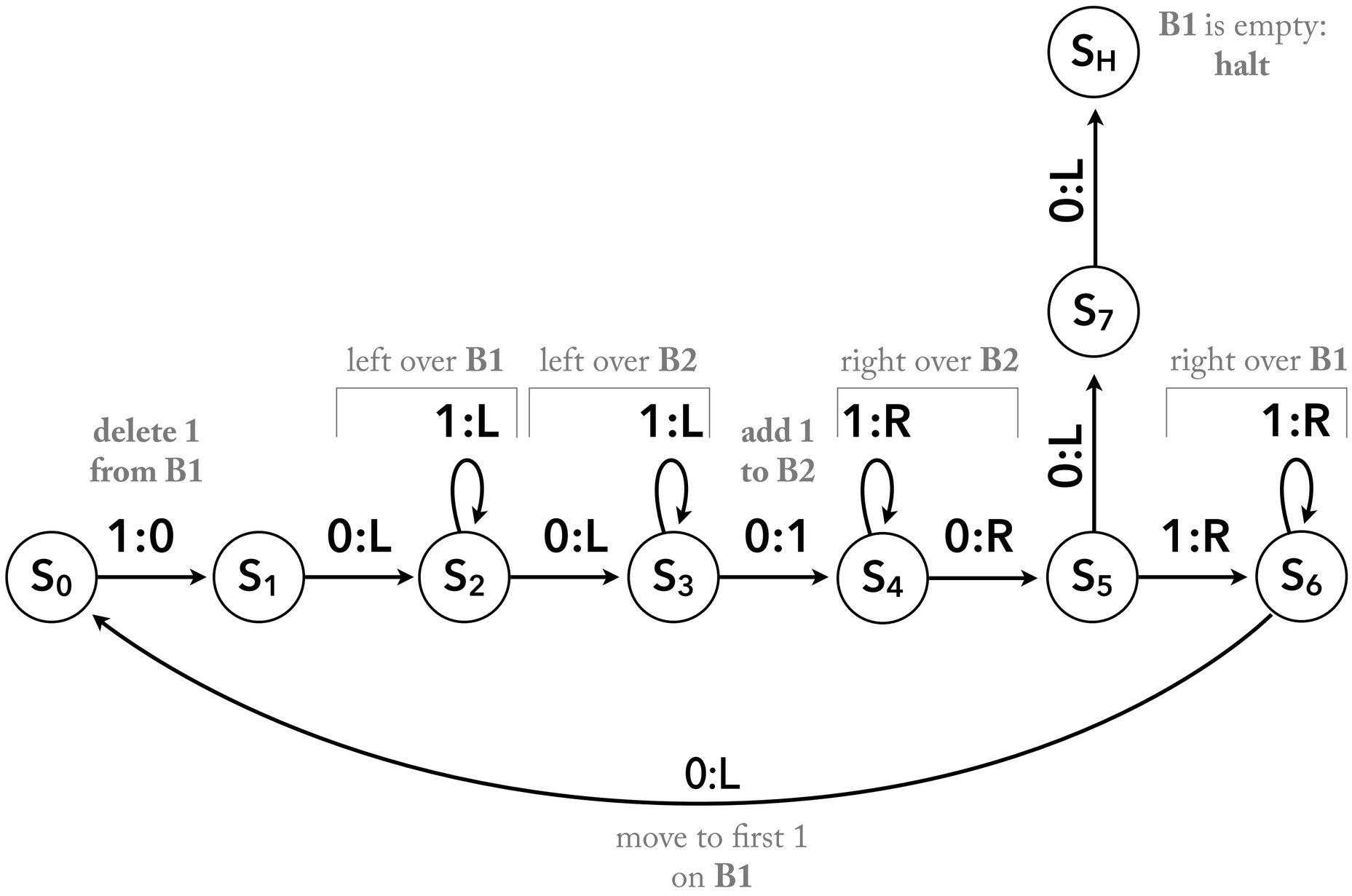
When a machine halts, computation ends. There is no going back.

- Machines do not have to halt. (They can go on forever.)
- A machine halts in a given state, if it is given an input and there are no arrows leaving that state for that input.
- If there are no lines leaving a state, the machine will halt in that state.

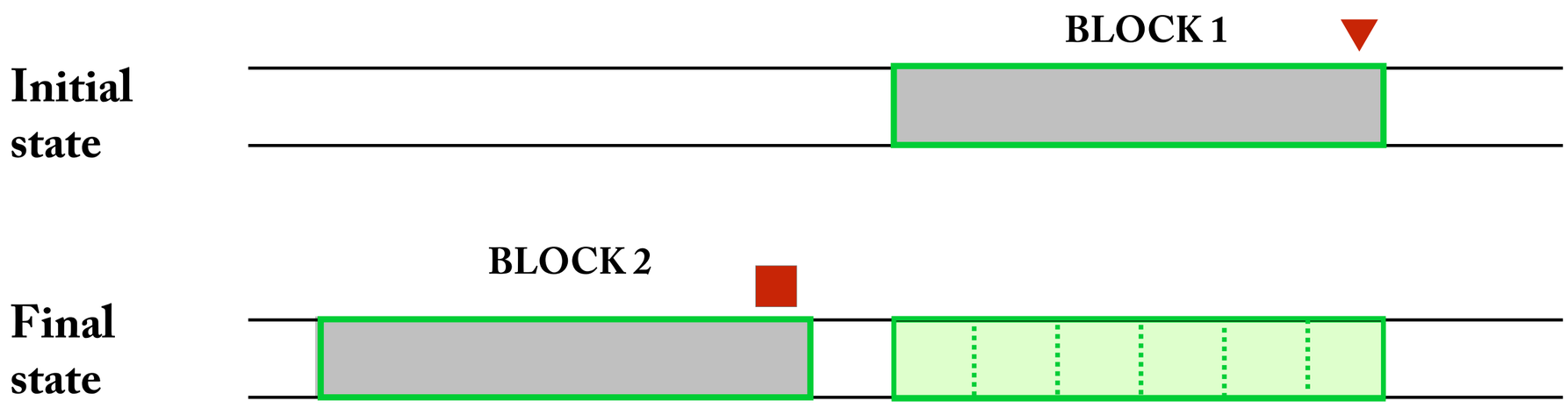


This machine halts in State 1.

Example: Cut and Paste



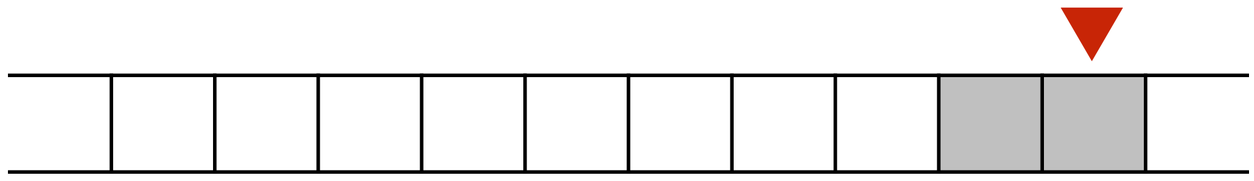
The goal is to shift a block of 1's from the Block 1 position to the Block 2 position, where Block 2 is to the left of the original Block 1 position, separated by 1 cell.



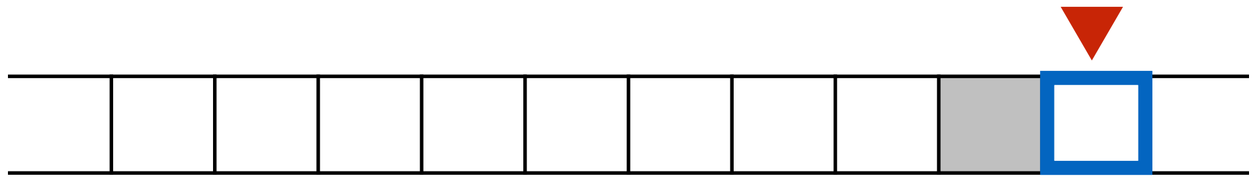
Cut-and-paste:

a) Remove the first 1 at the right edge of B1.
b) Move to the left edge of B1, then move two cells to the left.
c) Skip left over any 1's in B2 (initially there are none), and add a 1 at the end of B2.
d) Go back to B1.
e) Check if it is empty.
   If it is empty, move to B2 and halt.
   If it is not empty, go to the right edge of B1 and return to (a).

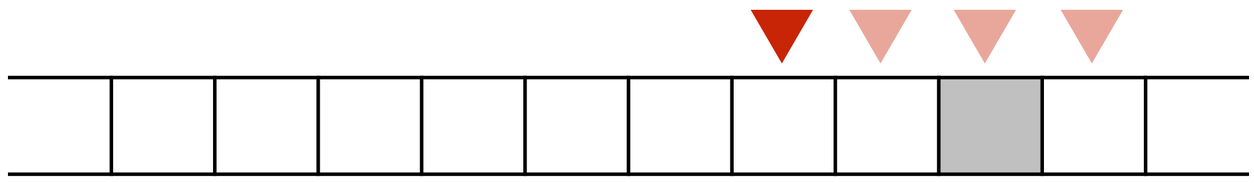
Standard position.



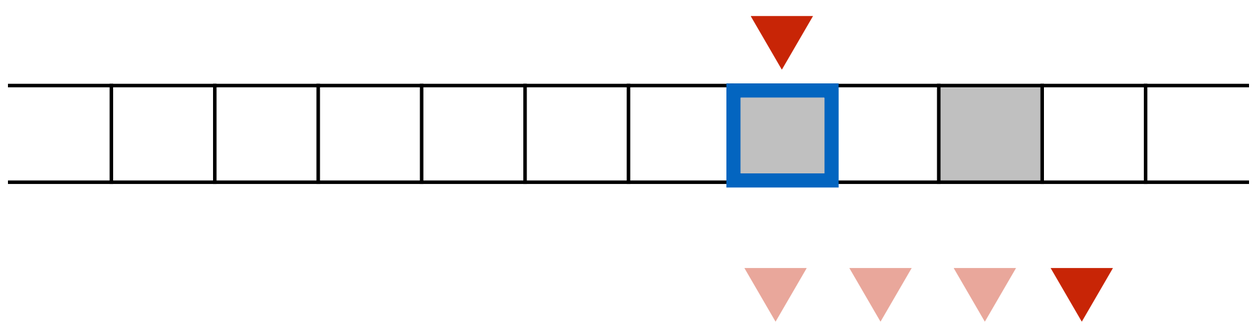
Delete 1 from B1.



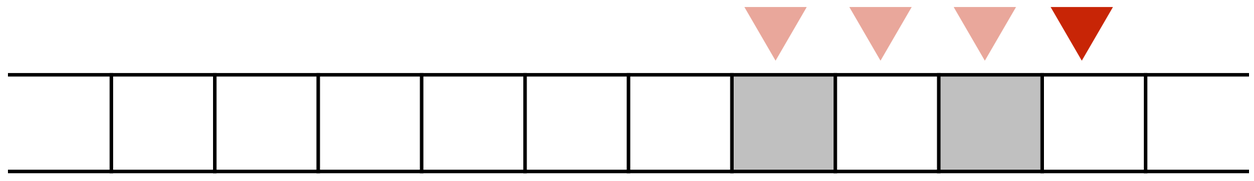
Move left to B2.



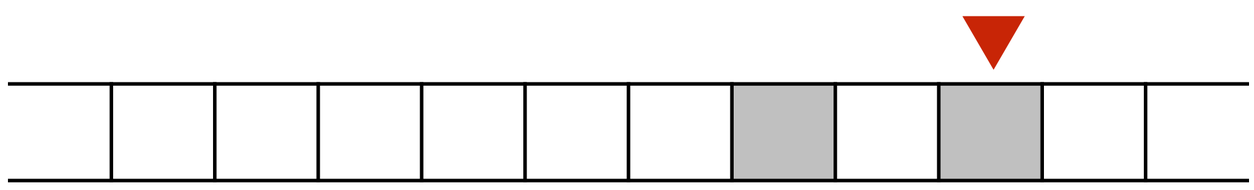
Add 1 to B2.



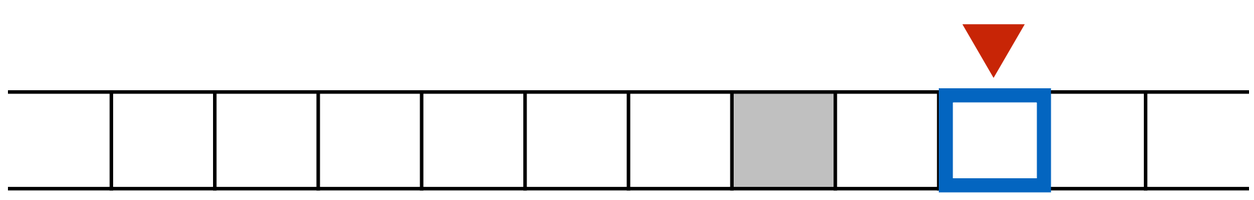
Move right to B1.



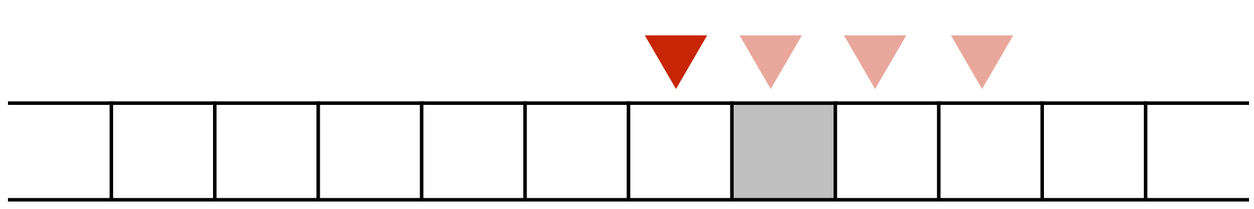
Move to first 1 of B1.



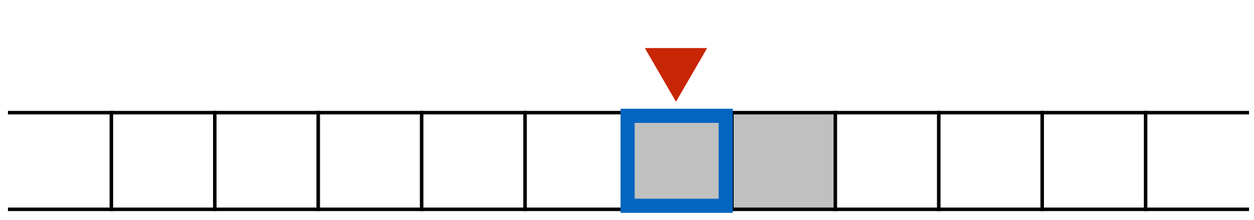
Delete 1 from B1.



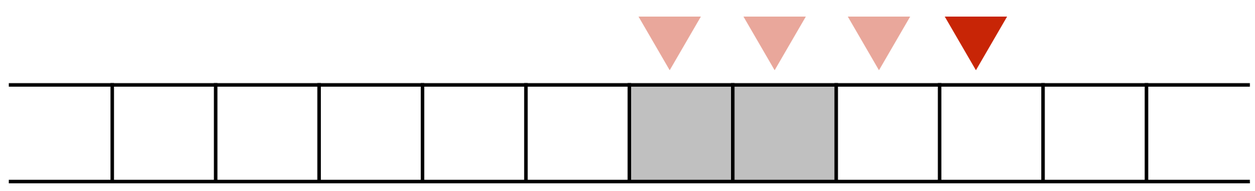
Move left to B2.



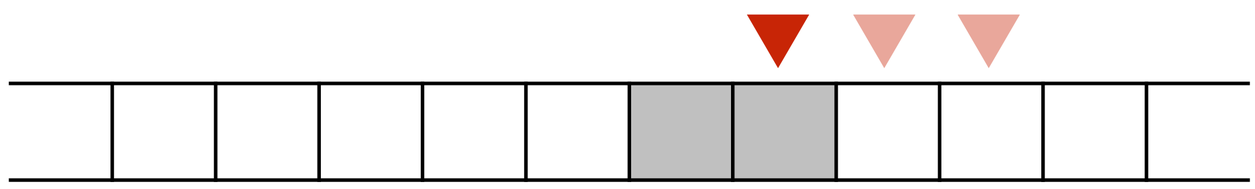
Add 1 to B2.



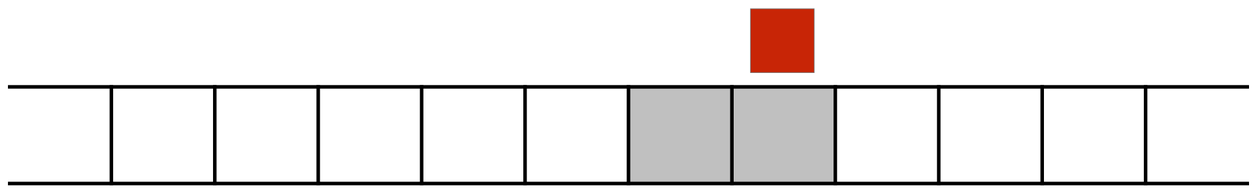
Move right to B1.



If B1 is empty, move to B2.



Halt.



In-Class Problems

Start on the right most cell (the "right edge") of the block in all problems.

1. **To the left gap:** Move left over a block of 1's, and halt on the first blank space to the left.
2. **To the left gap and back:** Move left over a block of 1's until you reach the gap, then return to the starting point and halt.
3. **To the left gap and +1:** Move left over a block of 1's, add a 1 at the end, then halt .
4. **To the left gap, +1, and back:** Move left over a block of 1's, add a 1 at the end, then return to the starting point and halt.
5. **Infinite expander:** Add a 1 at the left end of a block, then add a 1 at the right end of a block, and repeat forever.
6. **Cut-and-paste:** shift a block of 1's from the Block 1 position to the Block 2 position, where Block 2 is to the left of the original Block 1 position, separated by 1 cell.

